

Helmet Detection Using ConvNeXt

By Arinze Obidiegwu(23271892) and Steven Gerard Mascarenhas(24044407)

[Helmet Detection Using ConvNeXt](#)

[1. Introduction](#)

[2. Dataset & Preprocessing](#)

[3. Exploratory Data Analysis \(EDA\)](#)

[4. Model Architecture](#)

[5. Training & Results](#)

[6. Model Deployment](#)

[7. Challenges & Limitations](#)

[8. Conclusion](#)

[9. References](#)

1. Introduction

Helmet detection is a computer vision task designed to identify whether a person is wearing a helmet. This technology has significant applications in workplace safety, construction sites, and road traffic enforcement, where ensuring compliance with safety regulations is critical in preventing injuries and fatalities.

Traditional object detection approaches, such as Region-Based Convolutional Neural Networks (R-CNN) and its variants, rely on a two-stage process: first, identifying regions of interest, and second, classifying objects within those regions. While these methods provide high accuracy, they are computationally expensive and unsuitable for real-time applications where rapid detection is required.

In contrast, this project utilizes ConvNeXt, a modern deep learning model that achieves a balance between efficiency and accuracy. ConvNeXt is inspired by traditional CNN architectures but integrates design principles from Vision Transformers (ViTs) to enhance feature extraction, scalability, and training efficiency.

This report details the dataset preparation, preprocessing techniques, model architecture, training methodology, evaluation metrics, and deployment strategy used to develop the helmet detection system. The trained model is optimized using TorchScript, deployed via FastAPI, and integrated with a Streamlit-based web interface for real-time inference. To facilitate deployment and scalability, the system is further containerized using Docker.

1.1 Overview of Helmet Detection

The helmet detection system follows a classification-based approach, where an image is analyzed to determine whether a person is wearing a helmet. The detection pipeline consists of four primary stages:

1. Image Preprocessing

- Each image is resized to 224×224 pixels to match ConvNeXt's input dimensions.
- Images are normalized using ImageNet's mean and standard deviation to improve generalization and training stability.
- Data augmentation techniques such as random horizontal flipping, rotation, and color jittering are applied to improve the model's robustness to variations in lighting, pose, and background.

2. Feature Extraction & Classification

- ConvNeXt-Tiny, a pre-trained model on ImageNet, serves as the feature extractor.
- The model's final classification layer is modified to output three classes:
- Helmet → Person wearing a helmet (safe condition)
- Head → Person without a helmet, head visible (violation)
- Person → Person present, but helmet status unclear (uncertain case)

3. Inference & Prediction

- The trained model processes an input image and generates class predictions.
- The predicted class is mapped to its respective category:
 - **Helmet** → **Safe**
 - **Head** → **No Helmet**

4. Deployment & User Interaction

- The trained model is deployed as a REST API using FastAPI.
- A Streamlit-based frontend enables users to upload images and receive real-time helmet detection results.
- The system is containerized using Docker, ensuring scalability and ease of deployment in various environments.

By leveraging ConvNeXt, the system achieves high accuracy while maintaining computational efficiency, making it well-suited for real-time helmet detection applications in safety monitoring.

Key Advantages of Our Approach

Real-Time Efficiency → ConvNeXt enables faster inference compared to traditional CNN architectures.

Pre-Trained Weights → Using ImageNet-pretrained ConvNeXt improves generalization and accelerates convergence.

Scalability → The model architecture can be extended to detect additional Personal Protective Equipment (PPE), such as gloves, vests, and goggles.

2. Dataset & Preprocessing

2.1 Dataset Description

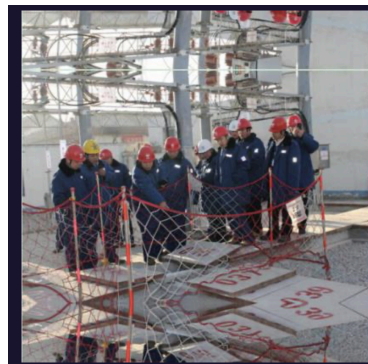
The dataset used for this project consists of helmet detection images annotated with bounding box labels in the Pascal VOC XML format. These annotations provide precise localization of objects within images, enabling supervised learning for object detection and classification.

To ensure robust model performance, the dataset was divided into the following subsets:

- **Training Set (80%)** → Used to train the ConvNeXt model.
- **Validation Set (20%)** → Used to assess model performance during training and fine-tuning.

The dataset contains three distinct classes, each representing a different helmet usage scenario:

- **Helmet** → The individual is wearing a helmet (safety-compliant).
- **Head** → The individual's head is visible without a helmet (safety violation).
- **Person** → A person is detected, but the helmet status is unclear (uncertain case).



2.2 Data Preprocessing Steps

To prepare the dataset for training, several preprocessing steps were applied to ensure data consistency, enhance model generalization, and improve training stability.

1. Annotations Conversion

- The original VOC XML annotations were parsed and converted into a structured CSV format.
- The CSV file contains image filenames, class labels, and bounding box coordinates (xmin, ymin, xmax, ymax).

2. Image Path Correction

- Image paths were corrected to reflect their respective train/validation directories, ensuring proper data organization.

3. Handling Missing Images

- Any non-existent or corrupted images were identified and removed from the dataset to prevent errors during model training.

4. Data Augmentation

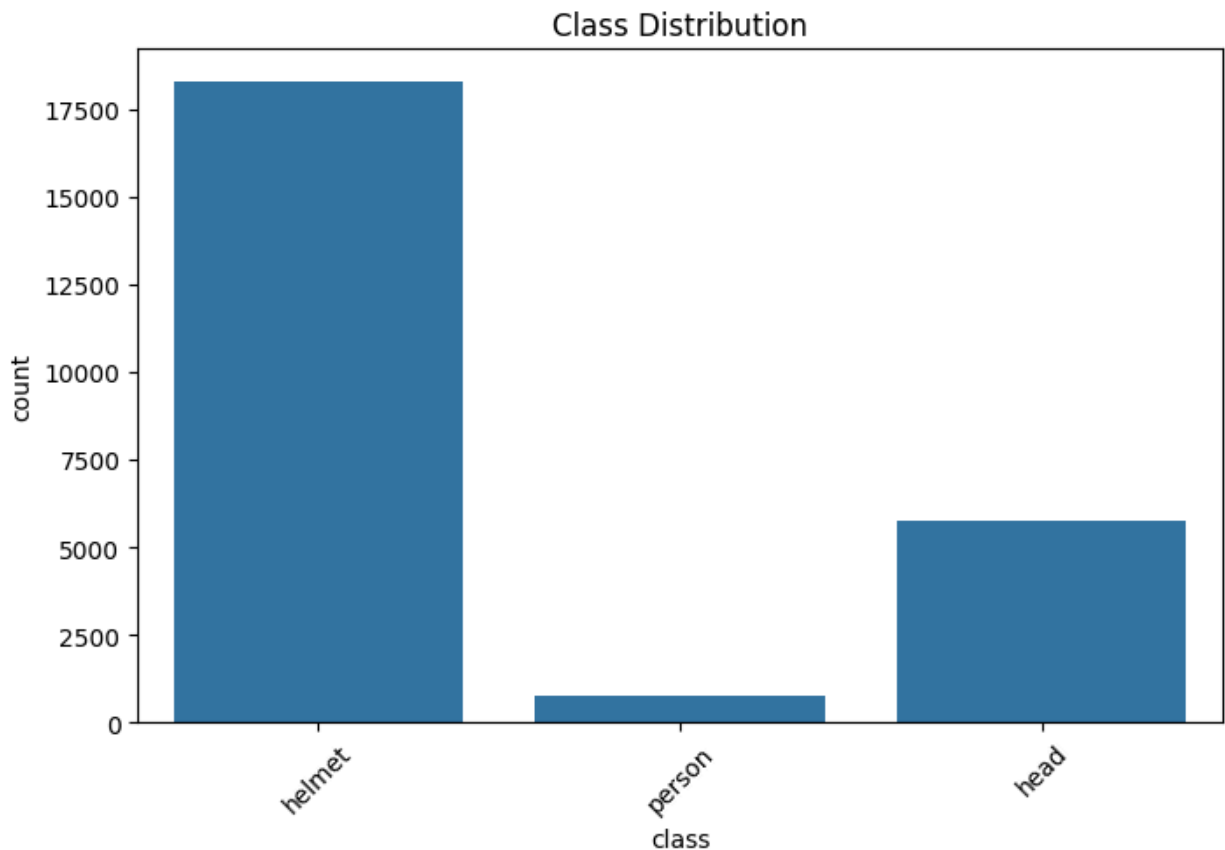
To improve model robustness and reduce overfitting, several augmentation techniques were applied to the training dataset:

Random Horizontal Flip → 50% chance of flipping images horizontally.

Random Rotation → $\pm 10^\circ$ random rotations to account for different viewing angles.

Color Jittering → Adjustments to brightness and contrast for lighting variations.

Normalization → Standardized using ImageNet mean and standard deviation to align with ConvNeXt's pretraining scheme.



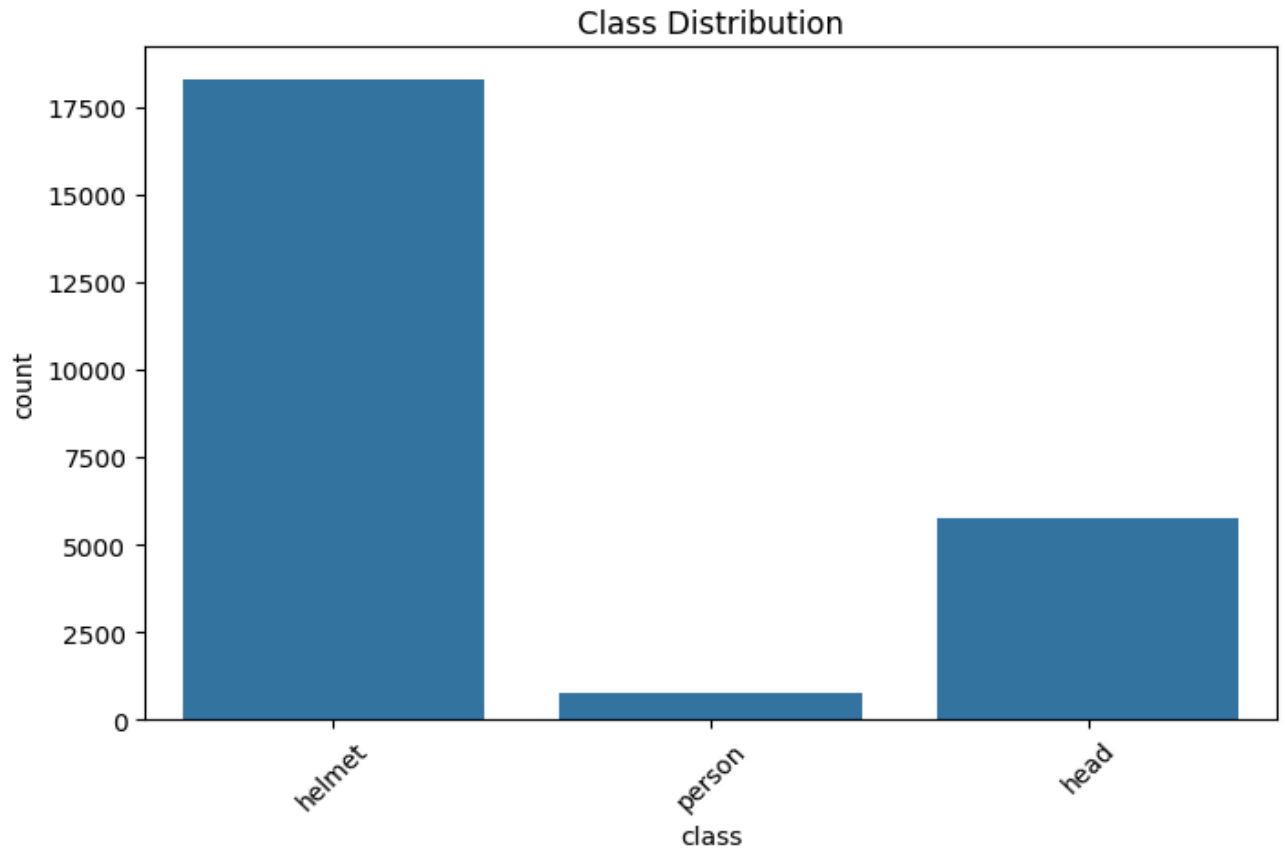
3. Exploratory Data Analysis (EDA)

Before training the model, an exploratory data analysis (EDA) was conducted to better understand dataset characteristics, identify potential biases, and guide preprocessing decisions. This section highlights key findings from the dataset analysis.

3.1 Class Distribution

An analysis of the dataset revealed class imbalance, with helmet images being significantly more frequent than head and person images. This imbalance could lead to biased model predictions, where the model might favor the majority class (helmet) over the others.

To visualize this distribution, a bar chart was generated, showing the number of images per class.



3.2 Bounding Box Area Analysis

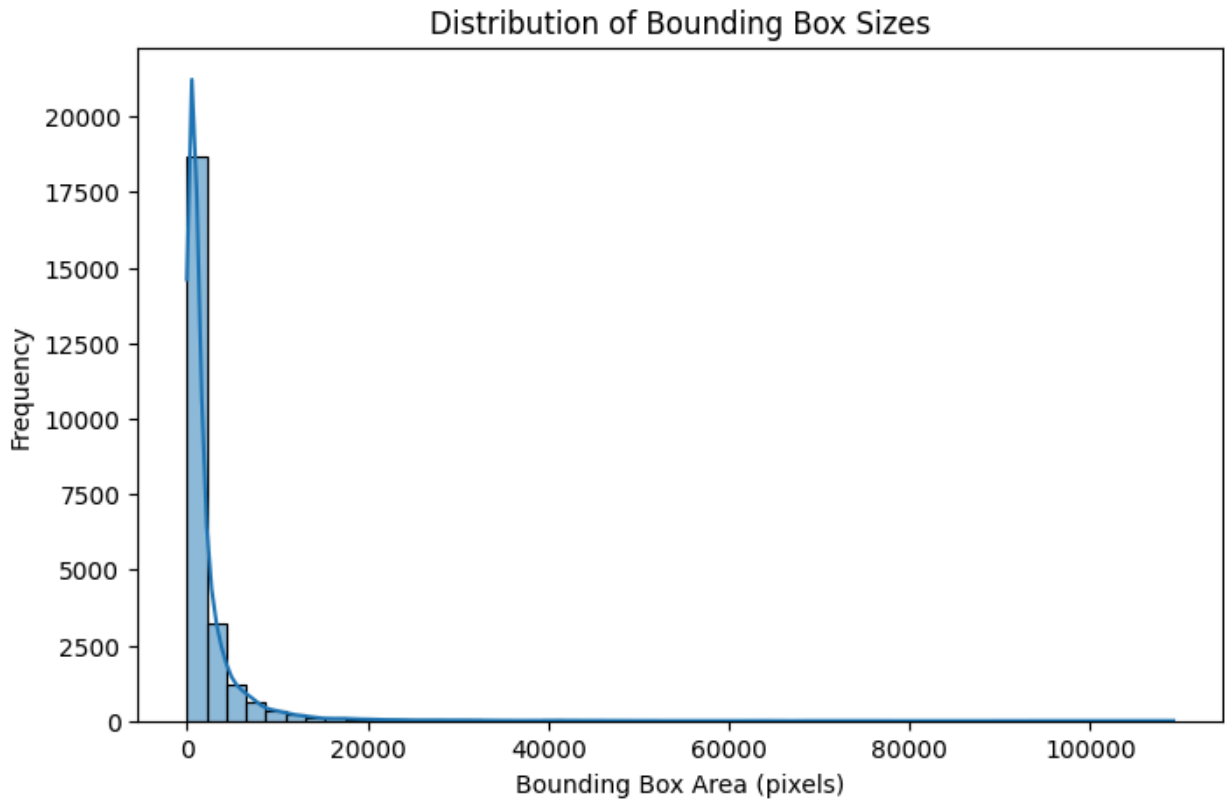
Bounding box sizes varied significantly across images, impacting the model's ability to detect objects of different scales. A histogram of bounding box areas was plotted to analyze the distribution of object sizes within the dataset.

Key observations:

The dataset contains both small and large bounding boxes, indicating variability in object distances.

Smaller bounding boxes may require higher-resolution feature maps for accurate detection.

Large bounding boxes may dominate training, requiring scaling adjustments to balance feature representation.



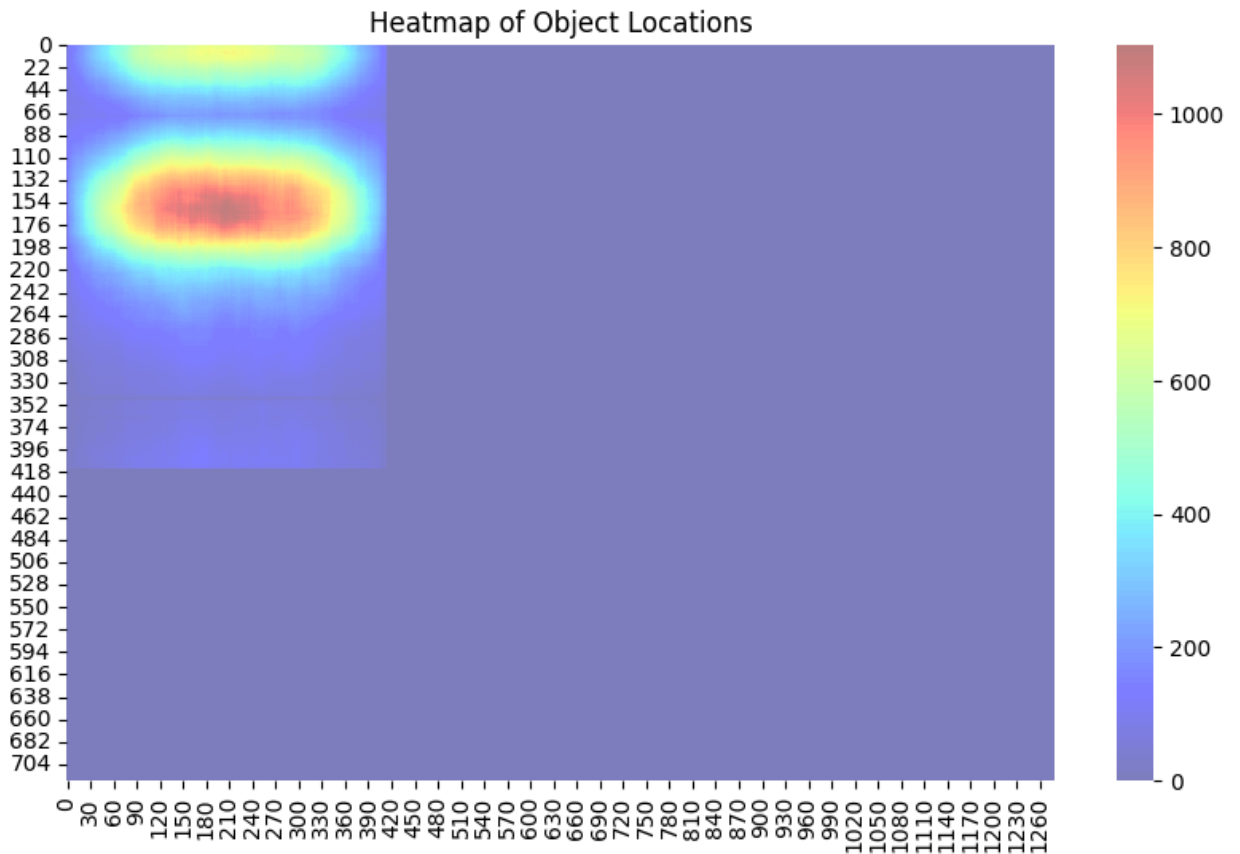
3.3 Object Density Heatmap

To identify common object locations within images, a heatmap of bounding box centers was generated. This visualization helps detect biases in object positioning.

Findings:

Objects are frequently centered in the image, suggesting a structured dataset.

Some images contain objects towards the edges, which may require augmentation strategies like cropping and shifting to improve model generalization.



3.4 Object Count Distribution

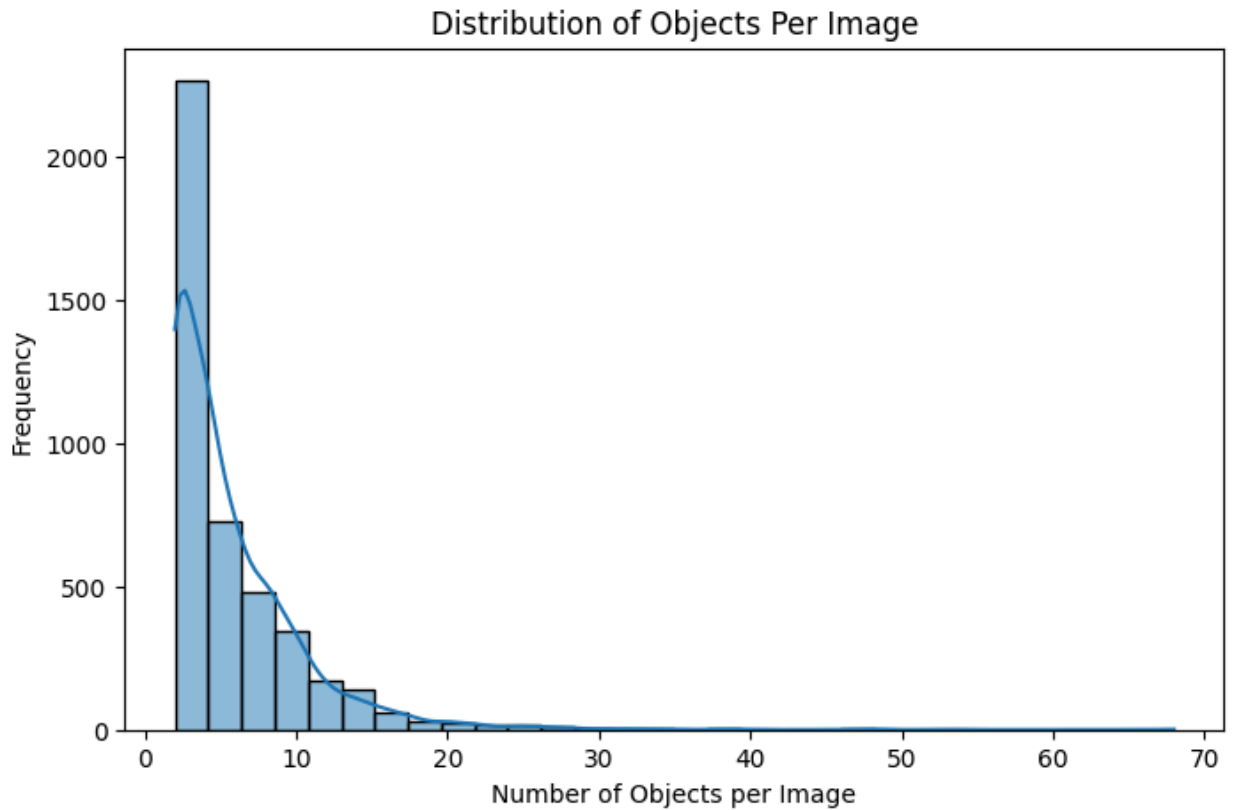
Understanding how many objects appear in each image is crucial for designing an effective detection model. Some images may contain multiple objects, while others may have only a single instance, affecting how the model generalizes to real-world scenarios.

Findings:

The dataset contains a mix of images with varying object counts per image.

A histogram of object counts reveals whether most images contain a single object or multiple objects.

If the dataset is dominated by single-object images, the model may struggle with multi-object detection and require augmentation strategies to mimic real-world diversity.



3.5 Helmet vs No-Helmet Distribution

Since this project focuses on helmet detection for safety monitoring, it's important to analyze the balance between images containing helmet wearers versus those without helmets. An imbalanced dataset may lead to biased predictions, where the model favors the more frequent class.

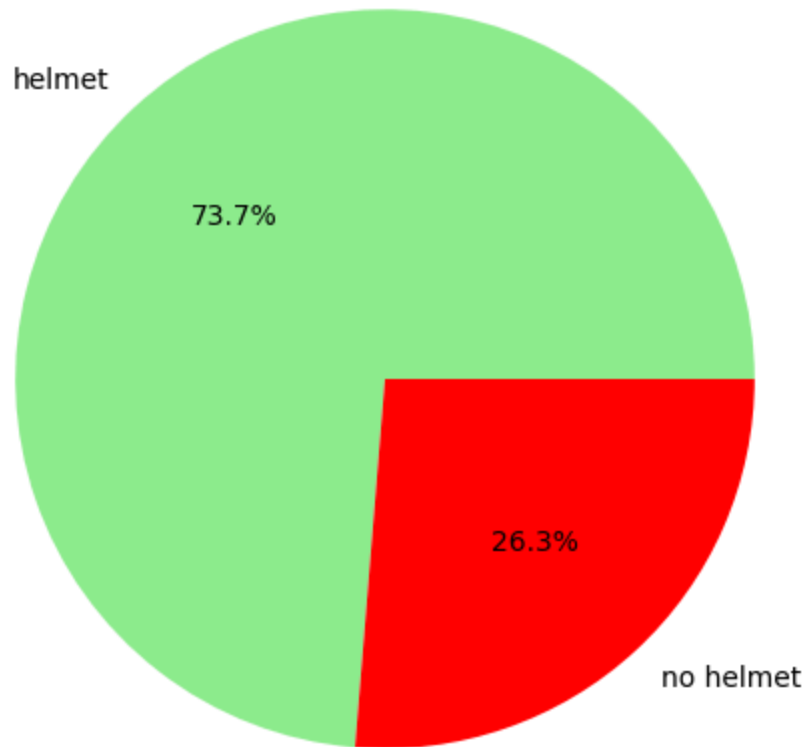
Findings:

A pie chart visualization shows the proportion of images containing helmet wearers versus non-wearers.

If helmets significantly outnumber non-helmet instances, techniques like class weighting or focal loss may be necessary to ensure fair learning.

Addressing class imbalance is crucial to reduce false negatives, ensuring the model does not overlook individuals without helmets.

Helmet vs No Helmet Distribution



3.6 Aspect Ratio Distribution of Bounding Boxes

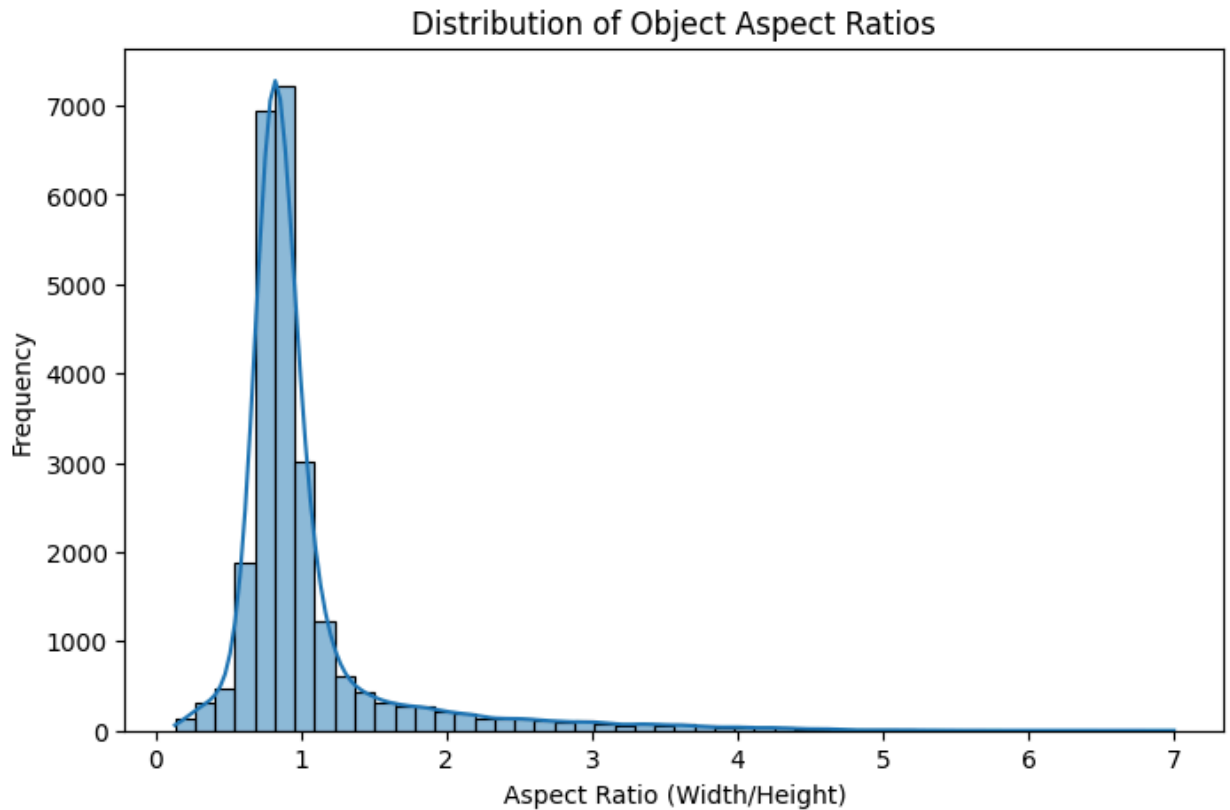
Bounding box dimensions play a critical role in how well the model detects objects, particularly in real-world images with varying object sizes and shapes. If bounding boxes are consistently square, the model may perform better. However, if aspect ratios are highly variable, the model might need adaptive anchor boxes to handle diverse object proportions.

Findings:

The aspect ratio histogram reveals whether bounding boxes have a dominant shape (e.g., square vs. rectangular).

If bounding boxes vary significantly, multi-scale feature extraction techniques (such as FPN - Feature Pyramid Networks) may be necessary.

Bounding box shape affects model performance, particularly for detecting small or elongated objects.



4. Model Architecture

4.1 Why ConvNeXt?

ConvNeXt is a modernized convolutional neural network (CNN) designed to bridge the gap between traditional CNNs and vision transformers. It enhances feature extraction while maintaining computational efficiency, making it well-suited for object detection tasks such as helmet detection.

Key Advantages of ConvNeXt:

Efficient Feature Extraction – Uses standard CNN layers optimized for hierarchical feature representation.

Transformer-Inspired Optimizations – Incorporates key architectural improvements, such as depth-wise convolutions and LayerNorm, improving model expressiveness.

Robust Transfer Learning – Pre-trained on ImageNet, allowing faster convergence and better generalization on new datasets.

By leveraging ConvNeXt-Tiny, this project benefits from state-of-the-art CNN performance while maintaining computational efficiency for real-time deployment.

4.2 Model Modifications

To adapt ConvNeXt for helmet detection, several modifications were applied to the pre-trained model:

Base Model: ConvNeXt-Tiny, pre-trained on ImageNet.

Final Classification Layer: Modified from 1000 classes → 3 classes (helmet, head, person).

Loss Function: CrossEntropyLoss, suitable for multi-class classification tasks.

These modifications ensure that the model is optimized for detecting helmets and distinguishing between different head states.

Model Summary

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 96, 56, 56]	4,704
LayerNorm2d-2	[-1, 96, 56, 56]	192
Conv2d-3	[-1, 96, 56, 56]	4,800
Permute-4	[-1, 56, 56, 96]	0
LayerNorm-5	[-1, 56, 56, 96]	192
Linear-6	[-1, 56, 56, 384]	37,248
GELU-7	[-1, 56, 56, 384]	0
Linear-8	[-1, 56, 56, 96]	36,960
Permute-9	[-1, 96, 56, 56]	0
StochasticDepth-10	[-1, 96, 56, 56]	0
CNBlock-11	[-1, 96, 56, 56]	0
Conv2d-12	[-1, 96, 56, 56]	4,800
Permute-13	[-1, 56, 56, 96]	0
LayerNorm-14	[-1, 56, 56, 96]	192

Linear-15	[-1, 56, 56, 384]	37,248
GELU-16	[-1, 56, 56, 384]	0
Linear-17	[-1, 56, 56, 96]	36,960
Permute-18	[-1, 96, 56, 56]	0
StochasticDepth-19	[-1, 96, 56, 56]	0
CNBlock-20	[-1, 96, 56, 56]	0
Conv2d-21	[-1, 96, 56, 56]	4,800
Permute-22	[-1, 56, 56, 96]	0
LayerNorm-23	[-1, 56, 56, 96]	192
Linear-24	[-1, 56, 56, 384]	37,248
GELU-25	[-1, 56, 56, 384]	0
Linear-26	[-1, 56, 56, 96]	36,960
Permute-27	[-1, 96, 56, 56]	0
StochasticDepth-28	[-1, 96, 56, 56]	0
CNBlock-29	[-1, 96, 56, 56]	0
LayerNorm2d-30	[-1, 96, 56, 56]	192
Conv2d-31	[-1, 192, 28, 28]	73,920
Conv2d-32	[-1, 192, 28, 28]	9,600
Permute-33	[-1, 28, 28, 192]	0
LayerNorm-34	[-1, 28, 28, 192]	384
Linear-35	[-1, 28, 28, 768]	148,224
GELU-36	[-1, 28, 28, 768]	0
Linear-37	[-1, 28, 28, 192]	147,648
Permute-38	[-1, 192, 28, 28]	0
StochasticDepth-39	[-1, 192, 28, 28]	0
CNBlock-40	[-1, 192, 28, 28]	0
Conv2d-41	[-1, 192, 28, 28]	9,600
Permute-42	[-1, 28, 28, 192]	0
LayerNorm-43	[-1, 28, 28, 192]	384
Linear-44	[-1, 28, 28, 768]	148,224
GELU-45	[-1, 28, 28, 768]	0
Linear-46	[-1, 28, 28, 192]	147,648
Permute-47	[-1, 192, 28, 28]	0
StochasticDepth-48	[-1, 192, 28, 28]	0
CNBlock-49	[-1, 192, 28, 28]	0
Conv2d-50	[-1, 192, 28, 28]	9,600
Permute-51	[-1, 28, 28, 192]	0
LayerNorm-52	[-1, 28, 28, 192]	384
Linear-53	[-1, 28, 28, 768]	148,224
GELU-54	[-1, 28, 28, 768]	0
Linear-55	[-1, 28, 28, 192]	147,648
Permute-56	[-1, 192, 28, 28]	0
StochasticDepth-57	[-1, 192, 28, 28]	0
CNBlock-58	[-1, 192, 28, 28]	0
LayerNorm2d-59	[-1, 192, 28, 28]	384
Conv2d-60	[-1, 384, 14, 14]	295,296
Conv2d-61	[-1, 384, 14, 14]	19,200

Permute-62	[-1, 14, 14, 384]	0
LayerNorm-63	[-1, 14, 14, 384]	768
Linear-64	[-1, 14, 14, 1536]	591,360
GELU-65	[-1, 14, 14, 1536]	0
Linear-66	[-1, 14, 14, 384]	590,208
Permute-67	[-1, 384, 14, 14]	0
StochasticDepth-68	[-1, 384, 14, 14]	0
CNBlock-69	[-1, 384, 14, 14]	0
Conv2d-70	[-1, 384, 14, 14]	19,200
Permute-71	[-1, 14, 14, 384]	0
LayerNorm-72	[-1, 14, 14, 384]	768
Linear-73	[-1, 14, 14, 1536]	591,360
GELU-74	[-1, 14, 14, 1536]	0
Linear-75	[-1, 14, 14, 384]	590,208
Permute-76	[-1, 384, 14, 14]	0
StochasticDepth-77	[-1, 384, 14, 14]	0
CNBlock-78	[-1, 384, 14, 14]	0
Conv2d-79	[-1, 384, 14, 14]	19,200
Permute-80	[-1, 14, 14, 384]	0
LayerNorm-81	[-1, 14, 14, 384]	768
Linear-82	[-1, 14, 14, 1536]	591,360
GELU-83	[-1, 14, 14, 1536]	0
Linear-84	[-1, 14, 14, 384]	590,208
Permute-85	[-1, 384, 14, 14]	0
StochasticDepth-86	[-1, 384, 14, 14]	0
CNBlock-87	[-1, 384, 14, 14]	0
Conv2d-88	[-1, 384, 14, 14]	19,200
Permute-89	[-1, 14, 14, 384]	0
LayerNorm-90	[-1, 14, 14, 384]	768
Linear-91	[-1, 14, 14, 1536]	591,360
GELU-92	[-1, 14, 14, 1536]	0
Linear-93	[-1, 14, 14, 384]	590,208
Permute-94	[-1, 384, 14, 14]	0
StochasticDepth-95	[-1, 384, 14, 14]	0
CNBlock-96	[-1, 384, 14, 14]	0
Conv2d-97	[-1, 384, 14, 14]	19,200
Permute-98	[-1, 14, 14, 384]	0
LayerNorm-99	[-1, 14, 14, 384]	768
Linear-100	[-1, 14, 14, 1536]	591,360
GELU-101	[-1, 14, 14, 1536]	0
Linear-102	[-1, 14, 14, 384]	590,208
Permute-103	[-1, 384, 14, 14]	0
StochasticDepth-104	[-1, 384, 14, 14]	0
CNBlock-105	[-1, 384, 14, 14]	0
Conv2d-106	[-1, 384, 14, 14]	19,200
Permute-107	[-1, 14, 14, 384]	0
LayerNorm-108	[-1, 14, 14, 384]	768

Linear-109	[-1, 14, 14, 1536]	591,360
GELU-110	[-1, 14, 14, 1536]	0
Linear-111	[-1, 14, 14, 384]	590,208
Permute-112	[-1, 384, 14, 14]	0
StochasticDepth-113	[-1, 384, 14, 14]	0
CNBlock-114	[-1, 384, 14, 14]	0
Conv2d-115	[-1, 384, 14, 14]	19,200
Permute-116	[-1, 14, 14, 384]	0
LayerNorm-117	[-1, 14, 14, 384]	768
Linear-118	[-1, 14, 14, 1536]	591,360
GELU-119	[-1, 14, 14, 1536]	0
Linear-120	[-1, 14, 14, 384]	590,208
Permute-121	[-1, 384, 14, 14]	0
StochasticDepth-122	[-1, 384, 14, 14]	0
CNBlock-123	[-1, 384, 14, 14]	0
Conv2d-124	[-1, 384, 14, 14]	19,200
Permute-125	[-1, 14, 14, 384]	0
LayerNorm-126	[-1, 14, 14, 384]	768
Linear-127	[-1, 14, 14, 1536]	591,360
GELU-128	[-1, 14, 14, 1536]	0
Linear-129	[-1, 14, 14, 384]	590,208
Permute-130	[-1, 384, 14, 14]	0
StochasticDepth-131	[-1, 384, 14, 14]	0
CNBlock-132	[-1, 384, 14, 14]	0
Conv2d-133	[-1, 384, 14, 14]	19,200
Permute-134	[-1, 14, 14, 384]	0
LayerNorm-135	[-1, 14, 14, 384]	768
Linear-136	[-1, 14, 14, 1536]	591,360
GELU-137	[-1, 14, 14, 1536]	0
Linear-138	[-1, 14, 14, 384]	590,208
Permute-139	[-1, 384, 14, 14]	0
StochasticDepth-140	[-1, 384, 14, 14]	0
CNBlock-141	[-1, 384, 14, 14]	0
LayerNorm2d-142	[-1, 384, 14, 14]	768
Conv2d-143	[-1, 768, 7, 7]	1,180,416
Conv2d-144	[-1, 768, 7, 7]	38,400
Permute-145	[-1, 7, 7, 768]	0
LayerNorm-146	[-1, 7, 7, 768]	1,536
Linear-147	[-1, 7, 7, 3072]	2,362,368
GELU-148	[-1, 7, 7, 3072]	0
Linear-149	[-1, 7, 7, 768]	2,360,064
Permute-150	[-1, 768, 7, 7]	0
StochasticDepth-151	[-1, 768, 7, 7]	0
CNBlock-152	[-1, 768, 7, 7]	0
Conv2d-153	[-1, 768, 7, 7]	38,400
Permute-154	[-1, 7, 7, 768]	0
LayerNorm-155	[-1, 7, 7, 768]	1,536

Linear-156	[-1, 7, 7, 3072]	2,362,368
GELU-157	[-1, 7, 7, 3072]	0
Linear-158	[-1, 7, 7, 768]	2,360,064
Permute-159	[-1, 768, 7, 7]	0
StochasticDepth-160	[-1, 768, 7, 7]	0
CNBlock-161	[-1, 768, 7, 7]	0
Conv2d-162	[-1, 768, 7, 7]	38,400
Permute-163	[-1, 7, 7, 768]	0
LayerNorm-164	[-1, 7, 7, 768]	1,536
Linear-165	[-1, 7, 7, 3072]	2,362,368
GELU-166	[-1, 7, 7, 3072]	0
Linear-167	[-1, 7, 7, 768]	2,360,064
Permute-168	[-1, 768, 7, 7]	0
StochasticDepth-169	[-1, 768, 7, 7]	0
CNBlock-170	[-1, 768, 7, 7]	0
AdaptiveAvgPool2d-171	[-1, 768, 1, 1]	0
LayerNorm2d-172	[-1, 768, 1, 1]	1,536
Flatten-173	[-1, 768]	0
Linear-174	[-1, 3]	2,307
=====		
Total params: 27,815,811		
Trainable params: 27,815,811		
Non-trainable params: 0		

Input size (MB): 0.57		
Forward/backward pass size (MB): 256.12		
Params size (MB): 106.11		
Estimated Total Size (MB): 362.80		

4.3 Training Configuration

The model was trained using the following hyperparameters to balance accuracy and efficiency:

Parameter	Value
Optimizer	Adam (Adaptive Moment Estimation)
Learning Rate	0.0001
Batch Size	32
Epochs	10

Loss Function	CrossEntropyLoss
GPU Acceleration	Enabled (CUDA for NVIDIA GPUs, MPS for Apple Silicon)

Training was accelerated using GPU to improve processing efficiency, reducing the overall training time while maintaining high accuracy.

5. Training & Results

5.1 Model Training Process

The ConvNeXt model was trained using mixed precision training, which optimizes computational efficiency by dynamically adjusting numerical precision. This technique helps reduce memory usage and increase training speed while maintaining accuracy.

To enhance model convergence, a learning rate scheduler was implemented, reducing the learning rate by a factor of 0.5 every five epochs. This approach stabilizes learning and prevents the model from making large, unstable updates in later epochs.

The training process was executed over 10 epochs, with:

- **Batch Size:** 32
- **Optimizer:** Adam (learning rate = **0.0001**)
- **Loss Function:** CrossEntropyLoss
- **Hardware:** GPU acceleration for efficient computation

5.2 Performance Metrics

The model was evaluated using standard classification metrics:

Accuracy: 91.13%

Precision, Recall, and F1-score were computed for each class to evaluate detection performance. These metrics provide deeper insights into model behavior:

- **Precision** indicates the proportion of correctly identified helmet instances.
- **Recall** measures the proportion of actual helmets correctly classified.
- **F1-score** balances precision and recall, ensuring a robust assessment of classification performance.

5.3 Confusion Matrix Analysis

To analyze misclassifications, a confusion matrix was generated.

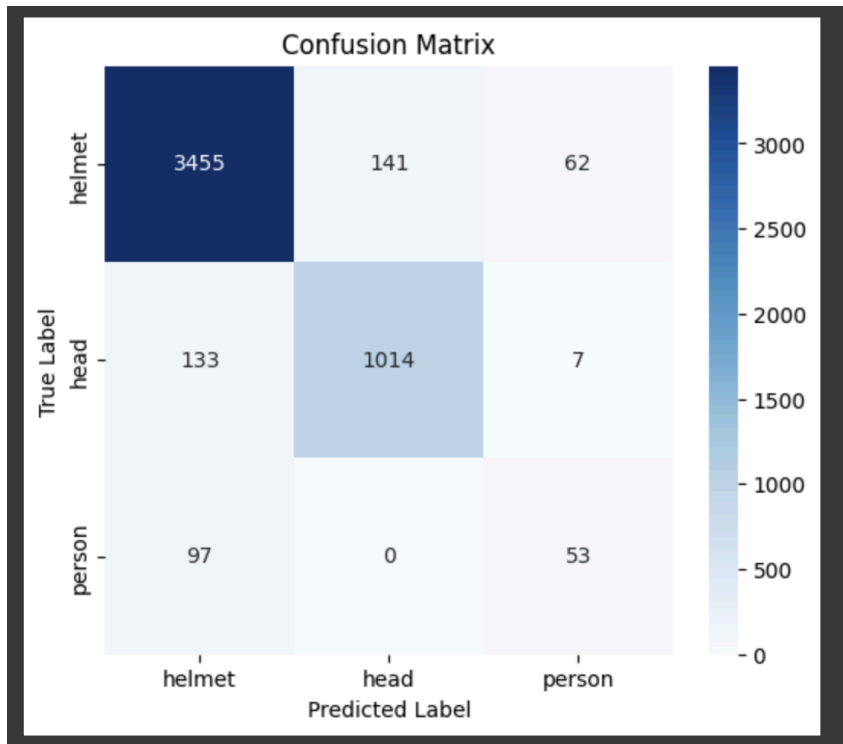
Findings:

Helmet detection was the most accurate, with minimal misclassification.

Misclassification occurred mainly between “head” and “person” classes, likely due to similarities in appearance.

The model occasionally struggled to differentiate between an uncovered head and an indistinct person, suggesting potential improvements through fine-tuning or additional

feature extraction techniques.



6. Model Deployment

6.1 TorchScript Conversion

To optimize the trained model for efficient deployment, it was converted into TorchScript (.pt) format. TorchScript enables:

Faster inference compared to standard PyTorch models.

Platform independence, making it deployable across different environments.

Optimized computation, reducing latency and improving performance in real-time applications.

The conversion was performed using `torch.jit.trace()`, which captures the model's computational graph with a sample input tensor, allowing it to run independently of the Python runtime.

6.2 FastAPI Backend

A FastAPI server was developed to serve the model via a REST API. The backend performs the following functions:

Accepts image uploads via a POST request at the /predict/ endpoint.

Preprocesses the image (resizing, normalization) before feeding it into the model.

Runs inference using the TorchScript model and returns the predicted class label.

This approach ensures a fast, lightweight, and scalable API, making it suitable for real-time helmet detection in web or mobile applications.

6.3 Streamlit Frontend

A Streamlit web interface was implemented for a user-friendly interaction with the model. The interface:

Allows users to upload images for classification.

Displays the uploaded image alongside the predicted class label.

Provides real-time results, ensuring an interactive experience.

The frontend directly communicates with the FastAPI backend, making a request each time a new image is uploaded.

Helmet Detection

Upload an image to check if a person is wearing a helmet.

Choose an image...



Drag and drop file here

Limit 200MB per file • JPG, PNG, JPEG

[Browse files](#)



OIP.jpeg 9.5KB



Uploaded Image

Prediction: No Helmet

Helmet Detection

Upload an image to check if a person is wearing a helmet.

Choose an image...



Drag and drop file here

Limit 200MB per file • JPG, PNG, JPEG

Browse files



download.jpeg 16.7KB



Uploaded Image

Prediction: **Helmet**

6.4 Dockerization

To facilitate easy deployment and scalability, both the backend (FastAPI) and frontend (Streamlit) were containerized using Docker.

Why Docker?

- Ensures consistent deployment across different environments.
- Allows for easy distribution and reproducibility of the application.
- Simplifies deployment on cloud servers or edge devices.

7. Challenges & Limitations

7.1 Dockerization

One of the primary challenges faced during training was class imbalance. The dataset contained significantly more images labeled as helmet, while the head and person classes were underrepresented. This imbalance can cause the model to be biased towards the majority class, leading to poorer generalization when detecting uncovered heads or ambiguous cases.

Potential Mitigation Strategies:

- **Data Augmentation:** Synthetic oversampling of underrepresented classes.
- **Re-weighting Loss Function:** Adjusting class weights in CrossEntropyLoss.
- **Collecting Additional Data:** Expanding the dataset with more diverse samples.

7.2 Bounding Box Variability

Bounding box annotations varied significantly across the dataset. Some annotations were too loose or too tight, leading to inconsistencies in training.

Challenges due to annotation issues:

- Bounding boxes that do not precisely capture helmets or heads may confuse the model.
- Inconsistent annotations affect feature extraction and object localization.

Possible Solutions:

- **Manual verification & cleaning** of bounding boxes.
- **Using automated outlier detection** to identify incorrect annotations.

7.3 Misclassification Between Head & Person

The most common source of misclassification was between head (no helmet) and person. This likely occurred due to:

- Visual similarities between a person's head and a full-body person without emphasis on the head.
- Bounding boxes that partially include the helmet or head, making it difficult to distinguish the classes.

Possible Enhancements:

- Fine-tuning the model on hard examples to improve class separation.
- Exploring alternative bounding box strategies, such as instance segmentation.

7.3 Deployment and Scaling

While the model runs efficiently in a local environment, deploying it at scale presents challenges:

Compute Requirements: Running the model on edge devices or low-power hardware may be resource-intensive.

API Latency: Processing large numbers of requests simultaneously may lead to bottlenecks.

Cloud Deployment Costs: Deploying on cloud GPUs can be expensive without efficient optimization.

Possible Future Improvements:

- Quantization & Pruning to reduce model size and inference time.
- Batch processing for API requests to improve scalability.
- Deploying on optimized cloud services (e.g., AWS Lambda)

[Github Link](#)

8. Conclusion

This project successfully developed a deep learning-based helmet detection system using ConvNeXt, achieving a high accuracy of 91%. The model was optimized for inference and deployed using FastAPI (backend) and Streamlit (frontend), making it accessible for real-world applications such as workplace safety enforcement, construction site monitoring, and traffic regulation compliance.

The implementation demonstrated the effectiveness of transfer learning by leveraging ImageNet-pretrained ConvNeXt and applying augmentation techniques to improve generalization. The use of TorchScript ensured efficient model inference, and Dockerization facilitated seamless deployment.

Possible Future Improvements

To further enhance the system's performance and applicability, the following improvements could be considered:

Addressing Class Imbalance:

- Expanding the dataset to include more diverse samples of the underrepresented classes (head and person).
- Applying data augmentation techniques tailored to minority classes.

Training on a Larger & More Diverse Dataset:

- Incorporating more helmet variations (e.g., different colors, shapes, and environments).
- Including harder samples, such as partially visible helmets or occlusions.

Enhancing Object Localization with Advanced Models:

- Transitioning from image classification to object detection for precise helmet localization.
- Exploring models like YOLO (You Only Look Once) or Faster R-CNN to detect helmets in complex scenes with multiple people.

Overall, this project demonstrated the feasibility of deep learning for helmet detection and laid the groundwork for further advancements in safety compliance automation.

9. References

1. Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., & Guo, B. (2022). *A ConvNet for the 2020s*. arXiv preprint arXiv:2201.03545. [Available at](#)
2. FastAPI Documentation. *FastAPI - Modern Web Framework for APIs with Python*. [Available at](#)
3. Streamlit Documentation. *Streamlit - The Fastest Way to Build and Share Data Apps*. [Available at](#)
4. PyTorch Documentation. *ConvNeXt-Tiny - PyTorch Vision Models*. [Available at](#)